

Código disponível em:

https://colab.research.google.com/drive/1lky4IP4i21_ivrFmnK8JKm5fgj0W_aJn?usp=sharing

Classificação ZERO SHOT

O que é classificação zero-shot ?

- É uma técnica que permite com que modelos de linguagem classifiquem textos em categorias nunca antes vistas, tendo por base apenas as descrições das categorias.
- Basicamente o modelo recebe apenas uma frase e um prompt que solicita com que a frase seja classificada em um conjunto de possíveis categorias.
 - Pode ser qualquer frase e um conjunto de categorias qualquer !!!!!

Exemplo dessa aula: Classificar filmes como "Romance", "Terror" ou "Comédia" em função de seu plot (resumo curto) no IMDb

- A base de dados contém o resumo de 110 filmes, selecionados do IMDb
 - Os 50 primeiros são resumos de filmes do gênero Romance
 - Os 50 seguintes do gênero Terror, também selecionados
 - Os 10 seguintes do gênero comédia (*com resumos menores*)
- Dessa forma, temos um problema de classificação multiclasse com 3 classes
- Faremos a classificação zero shot e a avaliação do desempenho com o F1-macro

```
# PASSO 1: importa a base de dados
import pandas as pd

df = pd.read_csv("https://github.com/edubd/bd/raw/refs/heads/main/resumos110.txt",
                 sep="|",
                 header = None,
                 names=["plot"])

df
```

```
...                                     plot
0    Pedro is a young man who earns his living as a...
1    Eva is an accidental witness of the death of a...
2    Amorosa is an emotional look into the life of ...
3    Los Angeles based architect Michael Lewis and ...
4    Dave is a computer programmer in his mid 20s w...
...                                     ...
105   In order to ruin a western town and steal thei...
```

```
# PASSO 2: converte o DataFrame para uma lista X
#           e gera uma lista Y com as classes reais
X = df['plot'].to_list()
Y = ['romance'] * 50 + ['horror'] * 50 + ['comedy'] * 10

print('X ->', X)
print('Y -> ', Y)
print()
print('exemplos:')
print(Y[0], X[0])      # horror
print(Y[50], X[50])    # romance
print(Y[100], X[100]) # comédia
```

```
X -> ['Pedro is a young man who earns his living as a street garbage collector. He lives in Medellin, on the shore of the river. On
Y -> ['romance', 'romance', 'romance', 'romance', 'romance', 'romance', 'romance', 'romance', 'romance', 'romance', 'romance', 'ro

exemplos:
romance Pedro is a young man who earns his living as a street garbage collector. He lives in Medellin, on the shore of the river. O
horror While a virus that causes the dead to reanimate brings the world to its knees, the scientist responsible entrusts his catacl
comedy In London, four very different people team up on a jewel heist, then try to double-cross one another for the loot, complicat
```

Classificação com o Modelo MoritzLaurer/deberta-v3-xsmall-zeroshot-v1.1-all-33

- Popular pretrained NLP model from Hugging Face used for zero-shot sequence classification.
- It allows you to classify text into arbitrary categories without additional training by framing classification as a natural language inference (NLI) task.
- 70 MB parâmetros

```
▶ # PASSO 3: importa um modelo para classificação ZERO SHOT
#           utilizando um Pipeline do Hugging Face
from transformers import pipeline
classifier = pipeline("zero-shot-classification",
                      model="MoritzLaurer/deberta-v3-xsmall-zeroshot-v1.1-all-33")

# algumas opções:
# MoritzLaurer/deberta-v3-xsmall-zeroshot-v1.1-all-33 (70M parâmetros)|
# MoritzLaurer/deberta-v3-base-zeroshot-v1.1-all-33 (200M parâmetros)
# facebook/bart-large-mnli (400M parâmetros)
```

```
*** Loading weights: 100%  202/202 [00:00<00:00, 6974.05it/s]
```

```

# PASSO 4: pede para o modelo classificar os resumos
# em uma das 3 classes candidatas

# dependendo do modelo escolhido, pode ser necessário modificar o código
# que recupera a classe predita a partir do resultado retornado pelo modelo

candidate_labels = ['romance', 'horror', 'comedy']

i = 0
Y_pred = []
for plot in X:
    i+=1
    predicao = classifier(plot, candidate_labels=candidate_labels, multi_label=False)
    max_pred = max(predicao['scores'])
    classe = predicao['labels'][0]
    Y_pred.append(classe)
    print(i, max_pred, classe, predicao)

... 1 0.8655248284339905 romance {'sequence': 'Pedro is a young man who earns his living as a street garbage collector. He live:
2 0.802717924118042 horror {'sequence': "Eva is an accidental witness of the death of a young girl, even more: she's got it
3 0.9649967551231384 romance {'sequence': 'Amorosa is an emotional look into the life of a young man who tries to find love
4 0.9798882603645325 romance {'sequence': 'Los Angeles based architect Michael Lewis and Russian language instructor Patric:
5 0.7213115692138672 romance {'sequence': 'Dave is a computer programmer in his mid 20s who works for a San Francisco tech (

```

Exemplo de resultado retornado:

{'sequence': 'Pedro is a young man who earns his living as a street garbage collector. He lives in Medellin, on the shore of the river. One day while working, he sees Juliana, a beautiful executive that seems to smile at him while she is shopping, unaware that she dropped her purse. Pedro runs to pick up the purse and returns it to her. She thanks him and leaves. After a very difficult day, Pedro cannot stop thinking about Juliana, who inspires him to dream, understand his reality, and gives him hope.',
 'labels': ['romance', 'comedy', 'horror'],
 'scores': [0.8655248284339905, 0.11419932544231415, 0.020275836810469627]}

```
print(len(Y), *Y)
print(len(Y_pred), *Y_pred)
```

Y

[illegible]

Y_pred

110 ['romance', 'horror', 'romance', 'romance', 'romance', 'comedy', 'romance', 'romance', 'romance',
 'romance', 'romance', 'romance', 'romance', 'romance', 'romance', 'horror', 'romance', 'romance',
 'romance', 'romance', 'romance', 'romance', 'romance', 'romance', 'romance', 'romance', 'horror',
 'romance', 'romance', 'comedy', 'comedy', 'romance', 'comedy', 'horror', 'romance', 'romance', 'horror',
 'comedy', 'horror', 'horror', 'romance', 'romance', 'romance', 'romance', 'romance', 'horror', 'horror',
 'horror', 'comedy', 'romance', 'horror', 'horror', 'romance', 'comedy', 'horror', 'horror', 'romance', 'horror',
 'horror', 'horror', 'horror', 'horror', 'horror', 'horror', 'comedy', 'horror', 'romance', 'horror', 'horror', 'horror',
 'romance', 'horror', 'horror', 'horror', 'horror', 'horror', 'horror', 'horror', 'comedy', 'horror', 'horror', 'horror',
 'horror', 'horror', 'horror', 'horror', 'horror', 'horror', 'horror', 'horror', 'horror', 'horror', 'horror', 'romance',
 'horror', 'horror', 'horror', 'horror', 'comedy', 'horror', 'comedy', 'comedy', 'horror', 'romance', 'comedy',
 'horror', 'romance', 'horror', 'horror', 'horror']

```

# PASSO 5: geração da matriz de confusão
from sklearn import metrics

candidate_labels.sort() # ordena as classes alfabeticamente

# matriz de confusão
mc = metrics.confusion_matrix(y_true = Y,
                              y_pred = Y_pred)

print('Matriz de Confusão\n', 'classes =', candidate_labels)
print(mc)

```

```

Matriz de Confusão
classes = ['comedy', 'horror', 'romance']
[[ 3  5  2]
 [ 4 41  5]
 [ 6 10 34]]

```

```

# PASSO 5: computa as médias macro e ponderada das medidas
print('Precision macro: ', metrics.precision_score(Y, Y_pred, average='macro'))
print('Precision ponderada: ', metrics.precision_score(Y, Y_pred, average='weighted'))
print()

print('Recall macro:: ', metrics.recall_score(Y, Y_pred, average='macro'))
print('Recall ponderada: ', metrics.recall_score(Y, Y_pred, average='weighted'))

print()
print('F1 macro: ', metrics.f1_score(Y, Y_pred, average='macro'))
print('F1 ponderada: ', metrics.f1_score(Y, Y_pred, average='weighted'))

print()
print('* valores por classe:')
print('precision: ', metrics.precision_score(Y, Y_pred, average=None)) # checando os valores por label
print('recall: ', metrics.recall_score(Y, Y_pred, average=None)) # checando os valores por label
print('f1:', metrics.f1_score(Y, Y_pred, average=None)) # checando os valores por label

print()
print('* Acurácia (não tem média macro ou ponderada...): ',
      metrics.accuracy_score(Y, Y_pred))
print()

```

```
# PRECISION[k] = VP[k] ÷ (VP[k] + FP[k])
# RECALL[k]     = VP[k] ÷ (VP[k] + FN[k])

# F1[k]         = (2 * VP[k]) ÷ (2 * VP[k] + FP + FN)
# fonte: https://scikit-learn.org/stable/modules/generated/sklearn.metrics.f1\_score.html
```

Precision macro: 0.5973934601983383
Precision ponderada: 0.7307113618089227

Recall macro:: 0.6
Recall ponderada: 0.7090909090909091

F1 macro:: 0.5939024060435053
F1 ponderada: 0.7150052572530013

* valores por classe:
precision: [0.23076923 0.73214286 0.82926829]
recall: [0.3 0.82 0.68]
f1: [0.26086957 0.77358491 0.74725275]

* Acurácia (não tem média macro ou ponderada...): 0.7090909090909091